FINAL PROJECT | INDUSTRY-SIMULATED AI PRODUCT WORKFLOW

Will this order arrive late?

Predicting late deliveries at checkout with a CrewAI Flow: a Data Analyst crew hands off to a Data Scientist crew

Matanel

Olist Brazilian e-commerce data | CrewAI + Gemini, scikit-learn, Streamlit, Flask, GitHub
Code: github.com/matnael332/olist-crewai-flow
Live dashboard: olist-late-delivery-1.streamlit.app

THE BUSINESS PROBLEM

Late deliveries destroy customer satisfaction

62.4%

of LATE orders get a 1-2 star review

9.2%

of ON-TIME orders get a 1-2 star review

6.8%

of 96,203 delivered orders arrived late



Goal: flag risky orders at checkout so operations can act before the customer is let down.

DATASET

Real marketplace data, cleaned and versioned

Source: Olist Brazilian E-Commerce, 9 tables (orders, items, payments, reviews, customers, products, sellers, geolocation, categories), pinned commit + sha256 manifest

Scale: 99,441 orders, 2016-09 to 2018-10

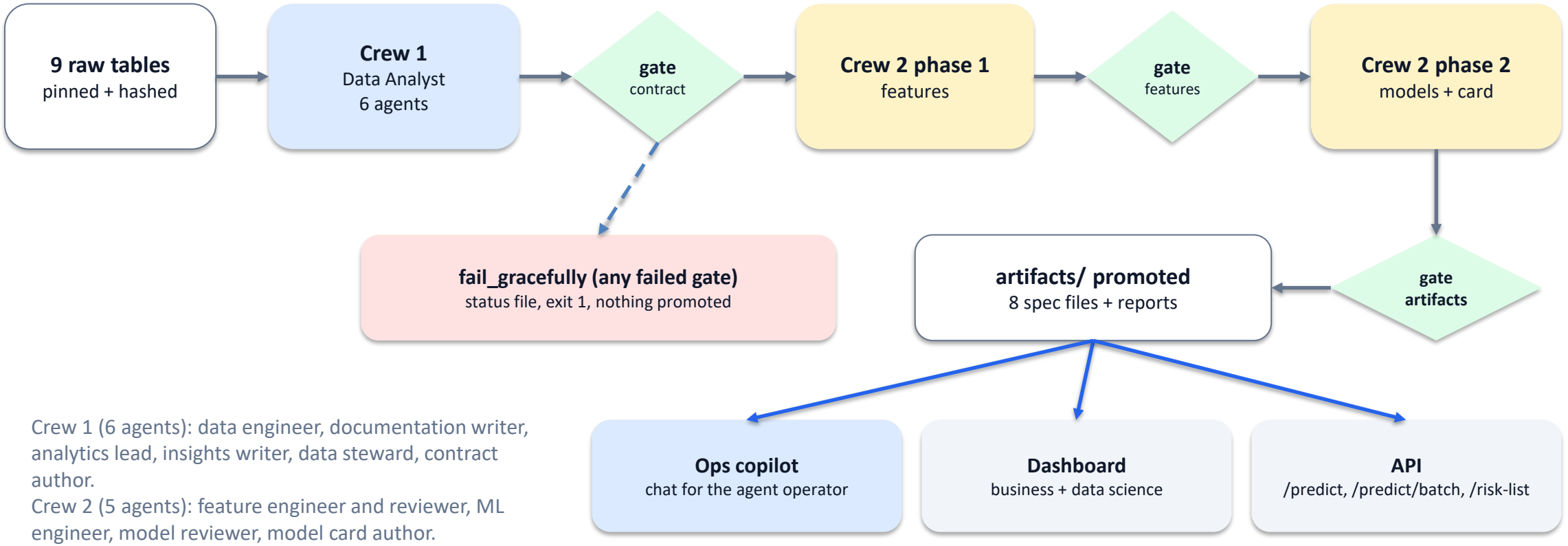
Real quality issues: non-delivered orders, missing delivery dates, out-of-sequence timestamps, products without category, duplicate reviews, invalid payments

Result: clean_data.csv with 96,203 delivered orders, one row per order, 38 documented columns

rule	rows removed
R1 keep delivered	2,963
R2 require delivery date	8
R3 purchase window	267
rows kept	**96,203**

ARCHITECTURE

Two AI crews, three quality gates, one product



Crew 1 (6 agents): data engineer, documentation writer, analytics lead, insights writer, data steward, contract author.
Crew 2 (5 agents): feature engineer and reviewer, ML engineer, model reviewer, model card author.

CREW 1 INSIGHTS

What happened: fast growth and a Black Friday spike



All 99,441 orders by purchase month, every status: 97.0% were delivered. November 2017, the Black Friday month, is the tallest bar.

800 to 7,544

orders per month, Jan 2017 to Nov 2017

1,176

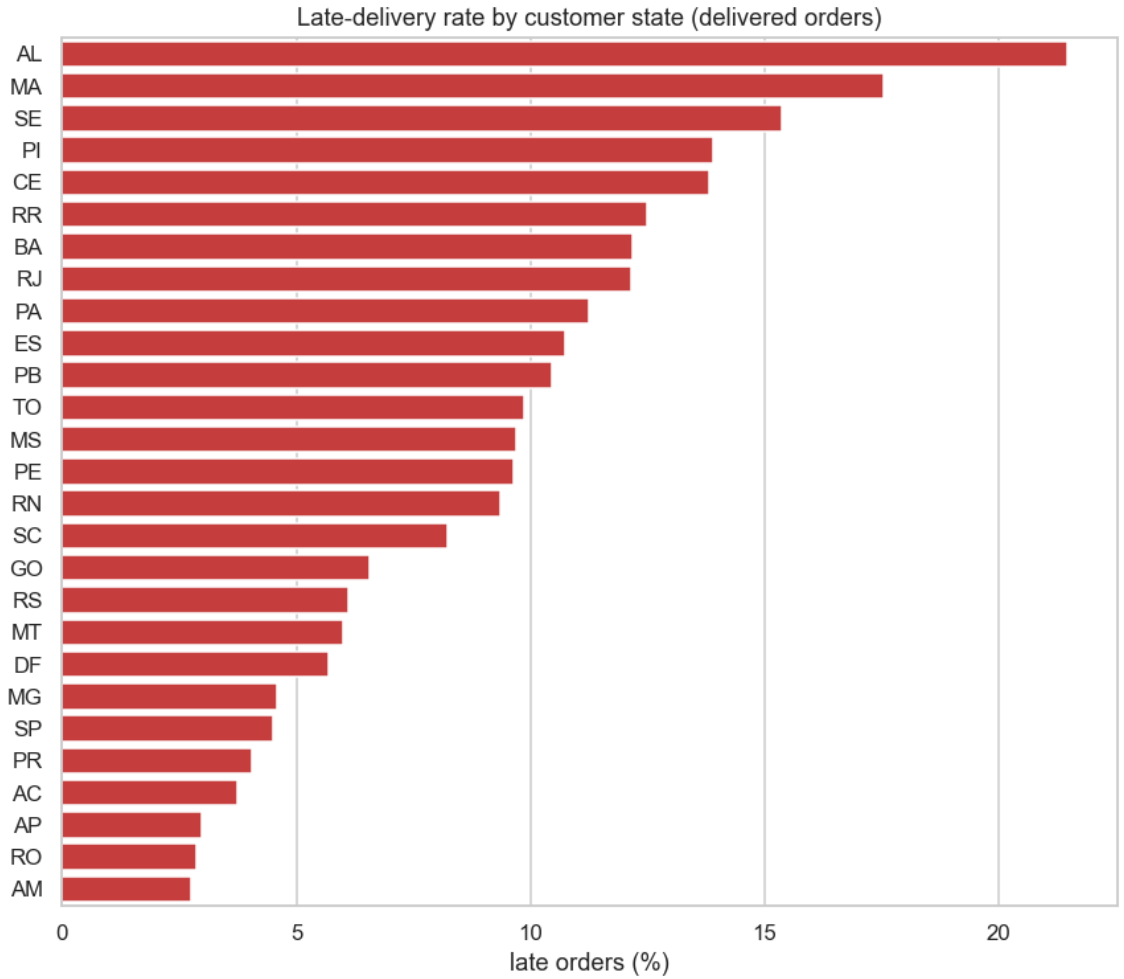
orders on 2017-11-24 (next busiest day: 499)

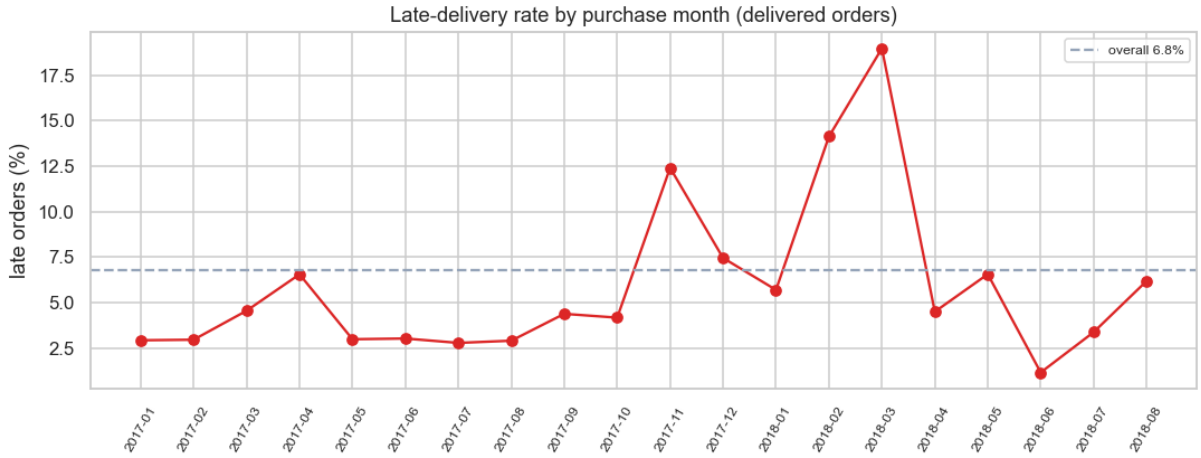
10 vs 24 days

median actual vs promised delivery time

CREW 1 INSIGHTS

Lateness depends on region and drifts over time





Worst: AL 21.5%, MA 17.5%, PI 13.9%

Best: MG 4.6%, SP 4.5%, PR 4.1%

Implication: evaluate on later months, never a random split

DATA CONTRACT

The dataset contract is the handoff

Schema: 38 columns with types, nullability, bounds and allowed values (27 states, 4 payment types, categories)

Constraints: 11 binding rules, e.g. is_late == (delivery_days > promised_days), one row per order

Target: is_late means delivered after the promised date, counted in calendar days, among delivered orders (6.8% late)

Required features: 14; forbidden post-purchase columns: 7 (no leakage)

Gate 1: the contract matches the cleaned data (21 checks)

Gate 2: every required feature exists before modeling (13 checks)

```
HANDOFF analyst->scientist
  contract_version=1.0
  contract_sha256=c330762f13cd
  rows=96203
  required=14 forbidden=7
```

The log line every run prints when the handoff passes; the same hash is stored in the feature spec, model metadata and model card.

CREW 2 DESIGN

Checkout-time features, time-aware evaluation

Features: 25 checkout-time features: basket, package, payment, geography (states, distance), promise and timing, category

Candidates: prior baseline, logistic regression, histogram gradient boosting (sklearn pipelines, seed 42)

Selection: time-series cross-validation on the train window, PR-AUC (imbalanced target)

Threshold: tuned on out-of-fold predictions only

Leakage guard: 7 post-purchase columns are forbidden; the features gate fails the run if one appears

split	purchase dates	orders	late rate
train	2017-01-01 to 2018-04-30	70,851	7.6%
test	2018-05-01 to 2018-07-31	19,001	3.8%
censored buffer	2018-08-01 to 2018-08-31	6,351	6.2%

August 2018 is held out as a censored buffer: the snapshot ends in mid-October 2018, so very late August orders may not have arrived yet.

RESULTS

The simpler model wins: 3x better than chance at the top

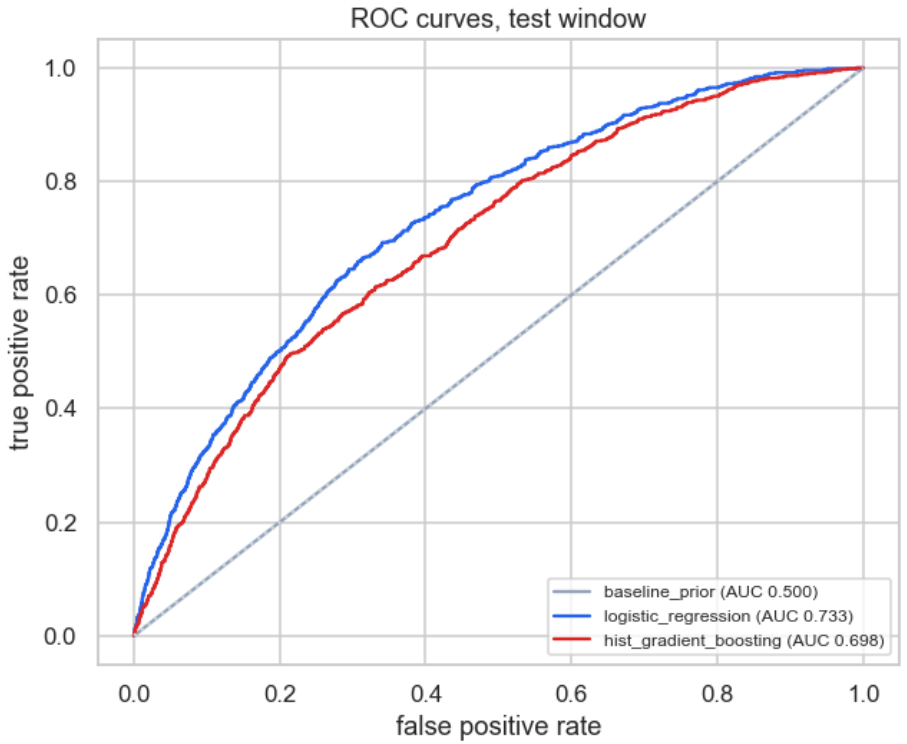
model	CV PR-AUC	test ROC-AUC	test PR-AUC	precision, top 10%
baseline prior	0.090	0.500	0.038	4.3%
hist gradient boosting	0.167	0.698	0.084	10.0%
logistic regression	**0.180**	**0.733**	**0.102**	**12.1%**

12.1%

late among the riskiest 10% (3.2x the 3.8% base rate)

0.733

test ROC-AUC (baseline 0.500)



Strongest signals: promised_days, customer_state, same_state, distance_km.

RELIABILITY

Engineered like a product: gates, failure, reproducibility

Gates: contract (21 checks), required features (13), artifacts (9); every check is logged

Graceful failure: 7 injectable faults; a failed gate writes run_status.json, exits 1 and promotes nothing

Reproducible: two fresh runs with different hash seeds give byte-identical data; the LLM run differs only in prose

Observable: pipeline.log records stages, gates, the handoff, crews, tools and every LLM call

Team workflow: one pull request per milestone, protected main, CI on every PR

```
$ olist_flow.run --no-llm --inject-fault missing_feature
FAULT INJECTED missing_feature
GATE gate_required_features required_features_present FAIL
GATE gate_required_features RESULT FAIL (11/13 checks)
RUN demo-fault FAILED at gate_required_features
run demo-fault: failed (exit 1), nothing promoted

$ olist_flow.run verify-repro
clean_data.csv        byte-identical  identical
features.csv          byte-identical  identical
metrics.json          byte-identical  identical
predictions_test.csv  byte-identical  identical
figures/*.png         same machine    11/11 identical
ALL DETERMINISTIC ARTIFACTS IDENTICAL
```

MODEL CARD

Responsible use: model card and ethics

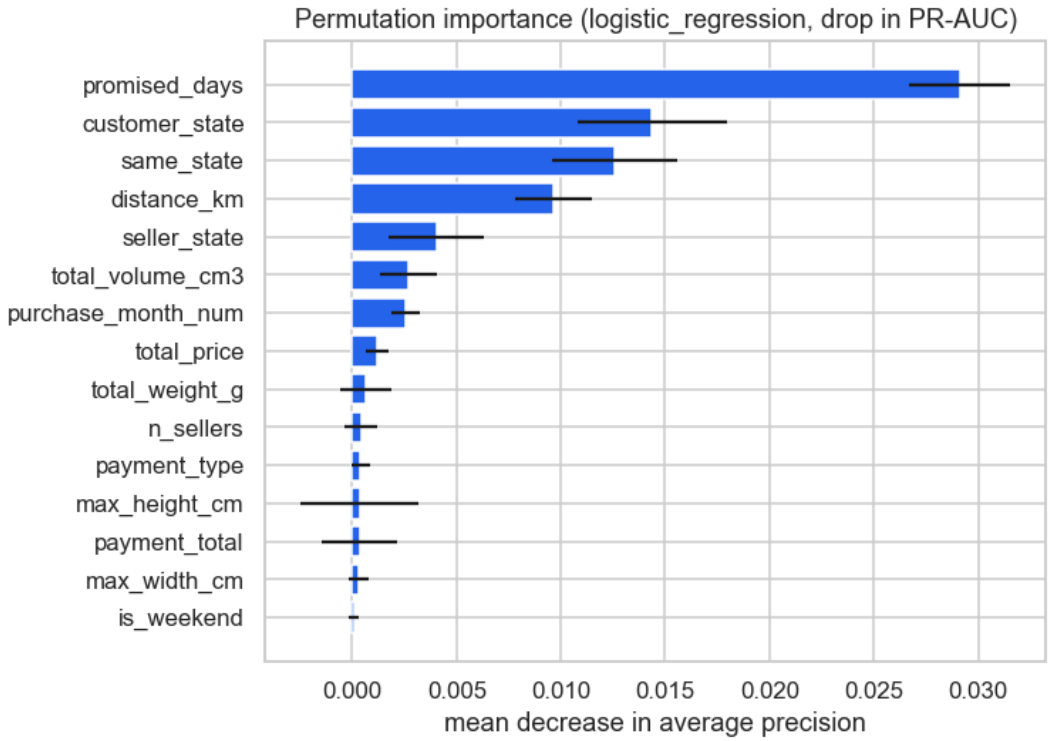
Purpose: prioritize risky orders for proactive action, never deny service or penalize customers or sellers

Limitations: delivered orders only; right-censoring; the late rate drifts, so the threshold must be re-tuned; 2017-2018 data

Regional bias: North and Northeast customers face more delays; interventions must improve their service

Privacy: pseudonymous ids; zip prefix plus timestamps are quasi-identifiers, so dashboards show aggregates only

License: Olist data for non-commercial use (CC BY-NC-SA 4.0 listing); review text is never used



The score leans on the promise length and on geography, so regional fairness is part of monitoring.

THE PRODUCT AND NEXT STEPS

From engine to product, and what comes next

Ops copilot: a chat for the team that operates the agents: runs, gates, model and data; scores a CSV, a replayed day or one order. Read-only, every number sourced

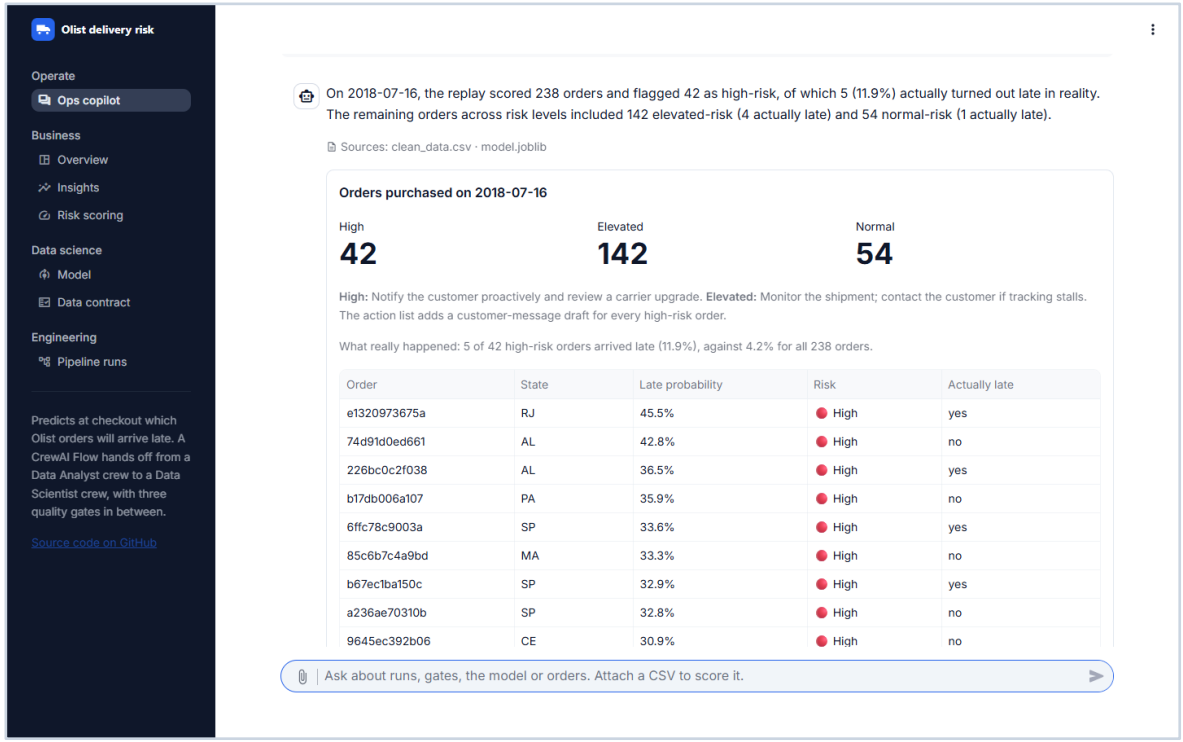
Dashboard and API: the same artifacts and one risk policy for people and systems

Run it: uv run kickoff (Gemini) or the --no-llm deterministic mode

Next: a Telegram bot and an 8:00 digest on the same copilot core

Next: send the customer messages and track handled orders

Next: seller-history features and a delivery-days regression



Live: olist-late-delivery-1.streamlit.app
Ops copilot replaying 2018-07-16, with what really happened